When each serving backend wins

A controlled comparison of **vLLM 0.31** and **llama.cpp (CUDA build)** on one GPU, from session `20261015T030427Z`.

Raw records: `results/raw/runs.jsonl` (filter with `python -m benchmax.cli report --session 20261015T030427Z`). Reproduce with `docs/reproducible-run-plan.md`.

Setup

GPU	NVIDIA GeForce RTX 4060 Laptop, 8188 MiB, driver 610.62, compute 8.9
Host	Windows 11, benchmark inside WSL2
Model	Qwen3-0.6B (base, not instruct)
llama.cpp	GGML_CUDA=ON, `CMAKE_CUDA_ARCHITECTURES=89`
vLLM	0.31.0, torch cu130
Replicates	n = 3, one discarded warmup sweep per cell

Results

Mean ± standard deviation over 3 replicates.

Single request (`w1_single_short`, 1 request, 128 output tokens)

Backend	Scheme	TTFT (ms)	TPOT (ms)	Output tok/s
llama.cpp	Q4_K_M	69.2 ± 99.4	**4.02 ± 0.14**	**223.8 ± 31.5**
llama.cpp	Q8_0	71.1 ± 104.1	4.38 ± 0.06	206.8 ± 29.4
vLLM	FP8 W8A8	**26.6 ± 4.6**	6.60 ± 0.28	147.9 ± 6.0

32-way concurrency (`w4_concurrent_32`)

Backend	Scheme	TTFT (ms)	TPOT (ms)	Output tok/s	Goodput tok/s
llama.cpp	Q4_K_M	3089.4 ± 89.6	11.40 ± 1.16	591.2 ± 22.7	147.8
llama.cpp	Q8_0	2657.2 ± 90.7	**10.29 ± 0.25**	660.8 ± 19.4	165.2
vLLM	FP8 W8A8	**161.1 ± 108.6**	10.51 ± 0.17	**2704.6 ± 206.9**	**2704.6**

What the numbers say

llama.cpp wins the single-user case

At one concurrent request llama.cpp delivers **~25% lower TPOT** and **~51% higher single-stream output throughput** than vLLM, and it does so with a much tighter relative spread. Its

decode loop is simply leaner: one sequence, no scheduler, no batch bookkeeping. If you are serving one user and care about how fast tokens appear, llama.cpp is ahead.

vLLM wins decisively once concurrency exists

At 32 concurrent requests vLLM produces **4.1x** the aggregate throughput of llama.cpp Q8_0 and cuts median TTFT by **~19x**. This is continuous batching doing what it exists for: llama.cpp processes sequences close to serially, so throughput barely moves between 1 and 32 requests (223.8 → 660.8 tok/s) while per-request TTFT degrades ~37x. vLLM amortises the decode step across the batch, so its TTFT at 32-way concurrency is only ~6x its single-request TTFT.

The practical consequence is visible in **goodput**, which requires both TTFT ≤ 1 s and TPOT ≤ 100 ms:

- vLLM: **2704.6 tok/s** — every request met both objectives
- llama.cpp Q8_0: 165.2 tok/s — roughly 6% of its own raw throughput qualifies

llama.cpp can post respectable aggregate throughput while failing a latency objective on almost every request. That is the failure mode the metric exists to catch, and it is invisible if you only look at tokens/second.

The measurement itself is noisy at n=3

llama.cpp's single-request TTFT has a standard deviation **larger than its mean** (± 99.4 ms on 69.2 ms). The first replicate is consistently the slow one. This is the warmup residual the harness already knows about — a single warmup sweep is not enough for llama.cpp — and it means the single-request TTFT column for llama.cpp should be read as "tens of milliseconds, occasionally 200".

vLLM's w4 TTFT spread is similarly wide (± 108.6 ms on 161.1 ms). The throughput ordering is robust across replicates; the precise latency values are not. Treat the ratio between backends as reliable and the absolute TTFT as a range.

Quantization effect is not separable from the backend

llama.cpp Q8_0 beats Q4_K_M on throughput under load (660.8 vs 591.2 tok/s), which is the opposite of the usual "smaller is faster" expectation and is a good reminder that these are different artifacts with different kernels, not the same weights at two bit-widths. **No configuration in this matrix serves identical weights on both backends**: llama.cpp consumes GGUF, vLLM consumes safetensors + FP8. Part of every delta above belongs to the quantizer and the artifact, not the serving engine. This is the single biggest limitation of the comparison and it is a property of the current releases, not of the harness.

When to pick which

Choose llama.cpp when: - concurrency is low, or you run one model per GPU - you want the lowest per-token latency for an interactive single stream - your hardware is a consumer card and you need the GGUF ecosystem (wide quantisation coverage, CPU offload fallback) - operational simplicity matters more than throughput

Choose vLLM when: - you serve many concurrent users - throughput per GPU is the binding constraint - you need continuous batching and paged attention to keep latency bounded - you want PagedAttention-style memory behaviour under variable load

The crossover is somewhere between 1 and 32 concurrent requests. This matrix only measures the two ends. The missing middle — `w3_concurrent_8` at n=3 — would locate it, and that is the most useful next measurement.

Quality axis

Generation-based exact match, dataset v1.0.0 (6 items), no logprob requirement:

Backend	Scheme	Exact match	Contains match	Failed	Token usage coverage
llama.cpp	Q4_K_M	0.000	0.167	0/6	1.0
llama.cpp	Q8_0	0.167	0.167	0/6	1.0
vLLM	FP8	0.167	0.000	0/6	1.0

Do not read these as a capability ranking. All three score at or near chance on a 6-item set, and the base (non-instruct) model largely fails to answer in the requested format. The metric is doing its job as a smoke test — it runs on both backends without logprobs, records raw answers, latency, token usage and failures — but it is not yet a capability signal. See issue #6.

Capability gaps recorded during this run

- **vLLM:** flashinfer sampler disabled automatically (`VLLM_USE_FLASHINFER_SAMPLER=0`). Its vendored CCCL headers are incompatible with the nvcc in the cu13 wheel, so its JIT sampling build fails and kills engine startup. Without this fallback the vLLM arm does not start at all. The sampler does not affect prompt/eval kernel selection, so the latency and throughput numbers above remain comparable — but this is a degraded configuration and is recorded as such in `startup_and_memory.csv`.

Limitations

- **n = 3**, on one consumer GPU shared with a running desktop. Enough to separate the backends by a large margin; not enough to resolve differences under ~10%, and several absolute latency values have standard deviations larger than their means.
- **Not weight-controlled** (see above).
- **No cloud arm.** No provider credentials were available.
- **Perplexity and MC accuracy are not comparable across backends**: vLLM scores via server echo logprobs (~18k tokens), llama.cpp offline from the identical GGUF (2048-token windows).
- **Two workloads only.** `w2_single_prefill` and `w3_concurrent_8` were not in this session.